

《数据科学》实验报告

|      |                 |      |                                                                                                   |    |          |
|------|-----------------|------|---------------------------------------------------------------------------------------------------|----|----------|
| 班级   | 003             | 姓名   | 史润                                                                                                | 学号 | 20241220 |
| 实验题目 | Python 基本用法实践练习 |      |                                                                                                   |    |          |
| 实验时间 | 2025-9-20       | 实验地点 | DS1408                                                                                            |    |          |
| 学年学期 | 2025-2026(1)    | 实验性质 | <input type="checkbox"/> 验证性 <input type="checkbox"/> 设计性 <input checked="" type="checkbox"/> 综合性 |    |          |

实验过程和结果

(1) 导入 keyword 模块，查看 Python 中的保留字/关键字。

代码：

```
import keyword

# 获取所有 Python 关键字
all_keywords = keyword.kwlist
# 打印关键字数量
print(f"Python 中的关键字共有 {len(all_keywords)} 个：")
# 打印所有关键字
print(all_keywords)
```

结果：

```
Python中的关键字共有 35 个：
['False', 'None', 'True', 'and', 'as', 'assert', 'async', 'await', 'break', 'class', 'continue', 'def', 'del', 'elif', 'else', 'except', 'finally', 'for', 'from', 'global', 'if', 'import', 'in', 'is', 'lambda', 'nonlocal', 'not', 'or', 'pass', 'raise', 'return', 'try', 'while', 'with', 'yield']
```

(2) 编写程序，输入一个自然数，输出它的二进制、八进制、十六进制形式。

代码：

```
# 获取用户输入的自然数
try:
    num = int(input("请输入一个自然数："))
    # 检查是否为自然数
    if num < 0:
        print("请输入一个非负整数！")
    else:
        # 进行进制转换
        binary = bin(num) # 二进制，结果以'0b'开头
        octal = oct(num) # 八进制，结果以'0o'开头
        hexadecimal = hex(num) # 十六进制，结果以'0x'开头
        # 输出转换结果
        print(f"\n十进制数 {num} 的各种进制表示:")
        print(f"二进制: {binary}")
        print(f"八进制: {octal}")
        print(f"十六进制: {hexadecimal}")

        # 额外输出不带前缀的形式
```

```

print("\n 不带前缀的表示:")
print(f"二进制: {binary[2:]}")
print(f"八进制: {octal[2:]}")
print(f"十六进制: {hexadecimal[2:].upper()}") # 十六进制通常用大写字母表示
except ValueError:
    print("输入错误! 请输入一个有效的自然数。")

```

结果:

```

请输入一个自然数: -3
请输入一个非负整数!

```

```

请输入一个自然数: 17

十进制数 17 的各种进制表示:
二进制: 0b10001
八进制: 0o21
十六进制: 0x11

不带前缀的表示:
二进制: 10001
八进制: 21
十六进制: 11

```

(3) 按要求完成以下列表的操作。

代码:

```

str = "123456789"
print(str) # 输出字符串
print(str[:-1]) # 输出第一个到倒数第二个的所有字符
print(str[0]) # 输出字符串第一个字符
print(str[2:5]) # 输出从第三个开始到第六个的字符 (不包含)
print(str[2:]) # 输出从第三个开始后的所有字符
print(str[1:5:2]) # 输出从第二个开始到第五个且每隔一个的字符 (步长为 2)
print(str * 2) # 输出字符串两次

```

结果:

```

123456789
12345678
1
345
3456789
24
123456789123456789

```

(4) 按要求完成以下对列表和元组的操作

代码:

```

# 初始列表
my_list = ["Chongqing", "Uni", 1923, 2023]
my_list[1] = "University" # 'Uni'改为'University'
my_list.insert(2, "year") # 'University'后添加 'year'
my_list.append("CHINA") # 列表最后添加 'CHINA'
print(my_list[-1].lower()) # 由列表输出'china' (将最后一个元素转为小写字母)
print(f"列表长度: {len(my_list)}") # 输出列表长度

```

```
my_tuple = tuple(my_list) # 将列表转换为元组
print(type(type(my_tuple))) # 用 type 打印出元组的类型

has_chongqing = "Chongqing" in my_tuple # 检查元组中是否有'Chongqing'
print(f"元组中是否包含'Chongqing': {has_chongqing}")
print("最终的元组:", my_tuple) # 打印最终的元组，查看结果
```

结果:

```
china
列表长度: 6
<class 'type'>
元组中是否包含'Chongqing': True
最终的元组: ('Chongqing', 'University', 'year', 1923, 2023, 'CHINA')
```

(5) 请按要求完成以下字典的操作

代码:

# 建立个人信息字典

```
my_info = {
    "姓名": "张三",
    "年龄": 20,
    "学号": "20241220",
    "年级": "大一",
    "班级": "01 班",
}
my_info["性别"] = "男" # 字典中加入性别信息
age_str = str(my_info["年龄"])
print(
    f"年龄（字符串类型）: {age_str}, 类型: {type(age_str)}"
) # 从字典中输出年龄（字符串类型）
student_id = my_info.pop("学号")
print(f"被删除的学号: {student_id}") # 从字典中输出学号，并从字典中删除
keys = list(my_info.keys())
third_key = keys[2]
print(f"第三个 key: {third_key}") # 从字典中输出第三个 key
my_info["班级"] = "数据科学 04 班" # 更新班级信息，"01 班" 改为 "数据科学 04 班"
print("操作后的字典:", my_info) # 打印最终的字典
```

结果:

```
年龄（字符串类型）: 20, 类型: <class 'str'>
被删除的学号: 20241220
第三个key: 年级
操作后的字典: {'姓名': '张三', '年龄': 20, '年级': '大一', '班级': '数据科学04班', '性别': '男'}
```

(6) 循环与条件判断

代码:

```
odd_sum = 0 # 初始化奇数和
even_sum = 0 # 初始化偶数和

# 遍历 1 到 50 的所有数字
for num in range(1, 51):
    if num % 2 == 0:
        even_sum += num # 累加偶数
```

```
    else:
        odd_sum += num # 累加奇数
print(f"1-50 内所有奇数的和为: {odd_sum}")
print(f"1-50 内所有偶数的和为: {even_sum}")
```

结果:

```
1-50内所有奇数的和为: 625
1-50内所有偶数的和为: 650
```

(7) 循环与数学运算。

代码:

```
print("所有的水仙花数 (3 位数) 有: ")
for num in range(100, 1000): # 遍历所有 3 位数 (100 到 999)
    hundreds = num // 100 # 分解出百位数字
    tens = (num // 10) % 10 # 分解出十位数字
    units = num % 10 # 分解出个位数字
    sum_of_cubes = hundreds**3 + tens**3 + units**3 # 计算每个位上的数字的 3
    次幂之和
    # 判断
    if sum_of_cubes == num:
        print(num)
```

结果:

```
所有的水仙花数 (3位数) 有:
153
370
371
407
```

(8) 遍历列表与字符串格式化。

代码:

```
name_list = ["Wangyifan", "Lisa", "Adam"] # 定义名字列表
# 遍历
for name in name_list:
    print(f"Hello, {name}!") # 使用 f-string 进行字符串格式化, 打印问候语
```

结果:

```
Hello, Wangyifan!
Hello, Lisa!
Hello, Adam!
```

(9) 列表操作与切片

代码:

```
numbers = [1, 5, 8, -17, 15, 25, 78, -5, 36, 50, 40] # 创建初始列表
A = sum(numbers[0:5]) # 计算列表中前 5 个数的和, 记为 A
B = sum(numbers[-5:]) # 计算列表中最后 5 个数的和, 记为 B
total = A + B
numbers.append(total) # 将结果附加到列表末尾
print(f"A (前 5 个数的和): {A}")
print(f"B (最后 5 个数的和): {B}")
print(f"A + B: {total}")
```

```
print("最终列表: ", numbers)
```

结果:

```
A (前5个数的和) : 12
B (最后5个数的和) : 199
A + B: 211
最终列表: [1, 5, 8, -17, 15, 25, 78, -5, 36, 50, 40, 211]
```

(10) 字典的创建与查询。

代码:

```
current_rating = {
    "照明灯具": "1-5 安培",
    "插座": "10-16 安培",
    "空调": "15-32 安培",
    "电热水器": "20-40 安培",
    "电磁炉": "15-30 安培",
} # 创建电气设备-额定电流范围字典
print("可用的设备类型: ", list(current_rating.keys()))
print("\n 查询结果: ")
print(f"空调的额定电流范围: {current_rating['空调']}") # 示例查询
print(f"电热水器的额定电流范围: {current_rating['电热水器']}")
# 交互式查询功能
print("\n----- 交互式查询 -----")
print("提示: 输入'退出'可以结束查询\n")
while True: # 循环查询功能
    device = input("请输入要查询的设备类型: ") # 获取用户输入
    if device == "退出":
        print("查询结束, 谢谢使用!") # 判断是否退出
        break
    if device in current_rating:
        print(f"{device}的额定电流范围是: {current_rating[device]}\n")
    else: # 检查设备是否在字典中并输出结果
        print(f"抱歉, 没有找到'{device}'的额定电流信息, 请重新输入\n")
```

结果:

```
可用的设备类型: ['照明灯具', '插座', '空调', '电热水器', '电磁炉']

查询结果:
空调的额定电流范围: 15-32 安培
电热水器的额定电流范围: 20-40 安培

----- 交互式查询 -----
提示: 输入'退出'可以结束查询

请输入要查询的设备类型: 插座
插座的额定电流范围是: 10-16 安培

请输入要查询的设备类型: 照明灯具
照明灯具的额定电流范围是: 1-5 安培

请输入要查询的设备类型: 手机
抱歉, 没有找到'手机'的额定电流信息, 请重新输入

请输入要查询的设备类型: 退出
查询结束, 谢谢使用!
```

(11) 字典查询的异常处理。

代码:

```
current_rating = {
    "照明灯具": "1-5 安培",
    "插座": "10-16 安培",
    "空调": "15-32 安培",
    "电热水器": "20-40 安培",
    "电磁炉": "15-30 安培",
} # 创建电气设备-额定电流范围字典
print("可用的设备类型: ", list(current_rating.keys()))

print("\n 查询结果: ")
print(f"空调的额定电流范围: {current_rating['空调']}") # 示例查询
print(f"电热水器的额定电流范围: {current_rating['电热水器']}")
# 交互式查询功能
print("\n----- 交互式查询 -----")
print("提示: 输入'退出'可以结束查询\n")
while True: # 循环查询功能
    device = input("请输入要查询的设备类型: ") # 获取用户输入
    if device == "退出":
        print("查询结束, 谢谢使用!") # 判断是否退出
        break

    rating = current_rating.get(device, "未找到") # 使用 get()方法安全查询, 避免 KeyError
    if rating != "未找到": # get()方法第二个参数为默认值, 当键不存在时返回该值
        print(f"{device}的额定电流范围是: {rating}\n")
    else:
        print(f"抱歉, 未找到该设备\n")
```

结果:

```
可用的设备类型: ['照明灯具', '插座', '空调', '电热水器', '电磁炉']

查询结果:
空调的额定电流范围: 15-32 安培
电热水器的额定电流范围: 20-40 安培

----- 交互式查询 -----
提示: 输入'退出'可以结束查询

请输入要查询的设备类型: 电热水器
电热水器的额定电流范围是: 20-40 安培

请输入要查询的设备类型: 电磁炉
电磁炉的额定电流范围是: 15-30 安培

请输入要查询的设备类型: 计算机
抱歉, 未找到该设备

请输入要查询的设备类型: 空调
空调的额定电流范围是: 15-32 安培

请输入要查询的设备类型: 退出
查询结束, 谢谢使用!
```

(12) 函数的定义与调用。

代码:

```
def factorial(n):
```

```
"""计算 n 的阶乘 (n!) """
result = 1
if n == 0 or n == 1:
    return 1 # 处理 n 为 0 或 1
for i in range(2, n + 1): # 迭代
    result *= i
return result # 定义计算阶乘的函数

# 计算 1! + 2! + ... + 10! 的总和
total = 0
for num in range(1, 11):
    total += factorial(num) # 调用阶乘函数并累加结果
    print(f"{num}! = {factorial(num)}") # 打印每个阶乘的结果, 方便查看中间过程
print(f"1! + 2! + 3! + ... + 10! = {total}") # 输出总和
结果:
```

```
1! = 1
2! = 2
3! = 6
4! = 24
5! = 120
6! = 720
7! = 5040
8! = 40320
9! = 362880
10! = 3628800
1! + 2! + 3! + ... + 10! = 4037913
```

(13) 赋值与拷贝的理解。

代码:

```
import copy

original = [
    100, # 不可变对象 (数字)
    "hello", # 不可变对象 (字符串)
    (1, 2, 3), # 不可变对象 (元组)
    [4, 5, 6], # 可变对象 (列表)
    {"a": 7, "b": 8}, # 可变对象 (字典)
] # 创建一个包含多种数据类型的嵌套列表

assigned = original # 使用赋值创建新变量
copied = copy.copy(original) # 使用浅拷贝创建新变量
print("=== 修改前 ===")
print("原始列表:", original)
print("赋值得到的列表:", assigned)
print("浅拷贝得到的列表:", copied)
print("原始与赋值的内存地址是否相同:", original is assigned)
print("原始与拷贝的内存地址是否相同:", original is copied)
print()
```

```
print("=== 修改原始列表中的元素 ===")

original[0] = 200 # 修改不可变元素数字（会创建新对象）
original[1] = "world" # 修改不可变元素字符串（会创建新对象）

original[3].append(7) # 修改可变元素列表
original[4]["c"] = 9 # 修改可变元素字典
print("修改后的原始列表:", original)
print("赋值得到的列表（assigned）的变化:", assigned)
print("浅拷贝得到的列表（copied）的变化:", copied)
print()
print("=== 结论分析 ===")
print("1. 赋值操作(=)创建的是引用，与原始对象指向同一内存地址")
print("2. 浅拷贝创建了新列表，但内部元素仍引用原始对象")
print("3. 不可变元素修改后：赋值对象会变化，拷贝对象不受影响")
print("4. 可变元素内部修改后：赋值对象和拷贝对象都会变化")
```

结果：

```
=== 修改前 ===
原始列表: [100, 'hello', (1, 2, 3), [4, 5, 6], {'a': 7, 'b': 8}]
赋值得到的列表: [100, 'hello', (1, 2, 3), [4, 5, 6], {'a': 7, 'b': 8}]
浅拷贝得到的列表: [100, 'hello', (1, 2, 3), [4, 5, 6], {'a': 7, 'b': 8}]
原始与赋值的内存地址是否相同: True
原始与拷贝的内存地址是否相同: False

=== 修改原始列表中的元素 ===
修改后的原始列表: [200, 'world', (1, 2, 3), [4, 5, 6, 7], {'a': 7, 'b': 8, 'c': 9}]
赋值得到的列表（assigned）的变化: [200, 'world', (1, 2, 3), [4, 5, 6, 7], {'a': 7, 'b': 8, 'c': 9}]
浅拷贝得到的列表（copied）的变化: [100, 'hello', (1, 2, 3), [4, 5, 6, 7], {'a': 7, 'b': 8, 'c': 9}]

=== 结论分析 ===
1. 赋值操作(=)创建的是引用，与原始对象指向同一内存地址
2. 浅拷贝创建了新列表，但内部元素仍引用原始对象
3. 不可变元素修改后：赋值对象会变化，拷贝对象不受影响
4. 可变元素内部修改后：赋值对象和拷贝对象都会变化
```